Genel Kodlama Kuralları ve Yazılım Geliştirme Rehberi

PayPorter Local LLM – PDF Okuyucu Test Dokümanı

Bu doküman, PayPorter için geliştirilen yerel LLM/RAG sisteminin PDF okuma, metin parçalama, anlamsal arama ve soru-cevap yeteneklerini test etmek amacıyla hazırlanmıştır. İçerik gerçek bir yazılım geliştirme rehberi biçimindedir ve farklı başlıklar, kurallar, kod örnekleri ve tablo içerir.

1. Okunabilir ve Anlaşılır Kod

Kod yalnızca bilgisayar tarafından çalıştırılmak için değil, insanlar tarafından okunmak için de yazılır. Değişken, fonksiyon ve sınıf isimleri yaptıkları işi açıkça ifade etmelidir. x, a1 veya tmp gibi belirsiz isimler yerine total_price, customer_id veya calculate_tax gibi açıklayıcı isimler tercih edilmelidir.

2. İsimlendirme Kuralları

Projede kullanılan isimlendirme biçimi tutarlı olmalıdır. Python'da değişken ve fonksiyonlar için snake_case, sınıflar için PascalCase yaygın olarak kullanılır. Sabit değerler büyük harflerle (MAX_RETRY_COUNT gibi) yazılabilir. Aynı projede farklı isimlendirme stillerinin rastgele karıştırılması bakım maliyetini artırır.

3. Fonksiyonların Tek Sorumluluğu

Bir fonksiyon mümkün olduğunca tek bir işi yapmalıdır. Veri doğrulama, veritabanına kayıt ve kullanıcıya mesaj üretme işlemlerinin tamamını tek fonksiyonda toplamak yerine görevler küçük fonksiyonlara ayrılmalıdır. Bu yaklaşım test yazmayı ve hata ayıklamayı kolaylaştırır.

Örnek:

```
def calculate_total(price, tax_rate):
    tax = price * tax_rate
    return price + tax
```

4. DRY – Kendini Tekrar Etme

Aynı kod bloğu birden fazla yerde tekrar ediyorsa ortak bir fonksiyona veya modüle taşınması düşünülmelidir. DRY (Don't Repeat Yourself) prensibi, bir değişiklik gerektiğinde aynı düzeltmenin birçok farklı noktada yapılmasını önler. Ancak gereksiz soyutlama da kodu zorlaştırabileceği için denge korunmalıdır.

5. Hata Yönetimi

Uygulamalar hataların oluşabileceğini varsaymalıdır. Dosya okuma, ağ bağlantısı, kullanıcı girdisi ve veritabanı işlemleri kontrollü şekilde ele alınmalıdır. Hataları sessizce yok saymak yerine anlamlı log mesajları üretilmeli ve mümkünse kullanıcıya güvenli bir hata mesajı gösterilmelidir.

6. Güvenlik Kuralları

Şifre, API anahtarı, token ve bağlantı bilgileri kaynak kod içine sabit olarak yazılmamalıdır. Gizli bilgiler environment variable veya güvenli secret yönetim sistemlerinde tutulmalıdır. Kullanıcı girdileri doğrulanmalı, yetkilendirme kontrolleri yalnızca arayüzde değil sunucu tarafında da uygulanmalıdır. SQL sorgularında parametreli sorgular kullanmak SQL injection riskini azaltır.

7. Loglama

Loglar sistem davranışını anlamak ve sorunları incelemek için önemlidir. INFO seviyesinde normal iş akışı, WARNING seviyesinde beklenmeyen fakat sistemi durdurmayan durumlar, ERROR seviyesinde ise başarısız işlemler kaydedilebilir. Şifre, kart bilgisi, erişim tokenı veya kişisel hassas veriler loglara açık biçimde yazılmamalıdır.

8. Yorum Satırları ve Dokümantasyon

Yorumlar kodun ne yaptığını tekrar etmek yerine neden o şekilde yazıldığını açıklamalıdır. Karmaşık iş kuralları, beklenmeyen teknik kararlar ve fonksiyonların giriş/çıkış davranışları belgelenmelidir. Güncelliğini kaybetmiş yorumlar yanlış yönlendirebileceği için kod değiştiğinde dokümantasyon da güncellenmelidir.

9. Git ve Versiyon Kontrolü

Değişiklikler küçük ve anlamlı commit'lere bölünmelidir. Commit mesajı yapılan değişikliği açıklamalıdır. Örneğin 'fix' yerine 'Fix PDF text extraction for empty pages' daha faydalıdır. Ana branch üzerinde doğrudan riskli geliştirmeler yapmak yerine feature branch kullanılması önerilir.

10. Test Yazımı

Kritik fonksiyonlar için otomatik testler oluşturulmalıdır. Normal kullanım senaryolarının yanında boş değer, yanlış veri tipi, sınır değer ve hata senaryoları da test edilmelidir. Bir test başarısız olduğunda hangi davranışın bozulduğu kolayca anlaşılabilmelidir.

11. Performans

Önce doğru ve okunabilir kod yazılmalı, ardından ölçüm yapılarak gerçek darboğazlar optimize edilmelidir. Gereksiz dosya okuma, tekrarlanan veritabanı sorguları ve büyük veri kümelerinin belleğe tamamen alınması performans sorunlarına yol açabilir.

12. Kod İnceleme Kontrol Listesi

Kod incelemesinde şu sorular sorulabilir: Kod anlaşılır mı? İsimler açıklayıcı mı? Tekrarlanan kod var mı? Hata durumları ele alınmış mı? Hassas bilgi açığa çıkıyor mu? Fonksiyon gereğinden uzun mu? Test edilebilir mi? Log mesajları yeterli mi? Değişiklik mevcut davranışları bozabilir mi?

13. Kısa Kod İnceleme Tablosu

Konu	Kötü Yaklaşım	Önerilen Yaklaşım
İsimlendirme	x = 150	order_total = 150
Gizli bilgi	API_KEY = 'abc123'	Environment variable kullan
SQL	String birleştirme	Parametreli sorgu
Hata	except: pass	Belirli exception + log
Commit	update	Add PDF validation before indexing

14. PDF Okuyucu İçin Örnek Test Soruları

1. Dokümana göre şifre ve API anahtarları kaynak kodda tutulmalı mı?
2. DRY prensibi ne anlama gelir?
3. Bir fonksiyonun tek sorumluluğa sahip olması neden önemlidir?
4. SQL injection riskini azaltmak için hangi yöntem öneriliyor?
5. ERROR ve WARNING log seviyeleri arasındaki temel fark nedir?
6. Git commit mesajı için verilen iyi örnek nedir?
7. Kod incelemesinde hangi güvenlik kontrolleri yapılmalıdır?
8. Performans optimizasyonuna ne zaman başlanması öneriliyor?

Test amacı: Sistem bu PDF'i indeksledikten sonra yukarıdaki sorulara yalnızca doküman bağlamını kullanarak cevap verebilmeli; ilgili parçaları doğru şekilde retrieve edebilmeli ve PDF dışındaki bilgileri cevaplara gereksiz biçimde karıştırmamalıdır.